ToggleMaster – Fase 1: Análise do Monolito e 12 Fatores

Tech Challenge – Pós-graduação em DevOps e Cloud Architecture (FIAP)

1. Execução local do monolito

1.1 O que é a aplicação

O **toggle-master-monolith** é uma API REST em Flask (servida por Gunicorn) para gerenciar *feature flags*. Os dados ficam em um banco PostgreSQL. A aplicação não tem interface gráfica: é consumida por requisições HTTP (curl, Postman, outros sistemas).

Método	Rota	Função
GET	`/health`	Verifica se a aplicação está no ar
POST	`/flags`	Cria uma flag (`name` , `is_enabled`)
GET	`/flags`	Lista todas as flags
GET	`/flags/<name>`	Consulta uma flag
PUT	`/flags/<name>`	Liga ou desliga uma flag (`is_enabled`)

1.2 Passo a passo executado

1. Fork do repositório `dougls/toggle-master-monolith` para a conta pessoal no GitHub.
2. Clone do fork pelo Git Bash: `git clone https://github.com/MarleyRacolto/toggle-master-monolith.git` .
3. Subida dos contêineres, dentro da pasta do projeto: `docker compose up --build` .
4. Validação em outro terminal: `curl http://localhost:5000/health` , com resposta `{"status":"ok"}` .
5. Encerramento com `Ctrl + C` e `docker compose down` .

O Docker Compose sobe dois contêineres: `app` (Flask + Gunicorn, porta 5000) e `db` (PostgreSQL 13). A aplicação espera o banco ficar saudável (`healthcheck` + `depends_on`), executa a inicialização do schema (tabela `flags`) e só então inicia o Gunicorn.

1.3 Problemas encontrados e soluções

Problema	Causa	Solução
`authentication required - personal access token is expired` ao baixar `postgres:13`	Sessão do Docker Hub com token expirado	`docker logout` e nova tentativa (a imagem é pública)
`entrypoint.sh: set: Illegal option -` e `app-1 exited with code 2`	O Git do Windows converteu o `entrypoint.sh` para final de linha CRLF; o contêiner Linux leu o `\r` como parte do comando	`sed -i 's/\r$//' entrypoint.sh` e criação do `.gitattributes` com `*.sh text eol=lf`, enviado ao fork

1.4 Resultado

A aplicação sobe localmente: o banco conecta, a tabela `flags` é criada, o Gunicorn escuta na porta 5000 e o `/health` responde `ok`.

2. Por que a aplicação é um monolito

2.1 Evidências no código

- **Uma única unidade de deploy:** um só `app.py`, uma só imagem Docker, um só processo de aplicação.
- **Responsabilidades misturadas no mesmo arquivo:** rotas HTTP, regras de validação, acesso a dados (SQL direto com `psycopg2`) e criação do schema (`init_db`).
- **Um único banco compartilhado:** todas as funções dependem da mesma tabela `flags` e da mesma conexão.
- **Acoplamento forte:** não há fronteiras entre domínios; qualquer mudança (por exemplo, adicionar autenticação ou auditoria) exige alterar e reimplantar toda a aplicação.
- **Escalabilidade só do conjunto:** não dá para escalar apenas a leitura de flags, por exemplo, sem replicar a aplicação inteira.

2.2 Vantagens para um MVP

- **Simplicidade:** um repositório, um build, um deploy. Fácil de entender e de rodar (um único `docker compose up`).

- **Desenvolvimento rápido:** sem contratos entre serviços, sem comunicação de rede interna, sem orquestração.
- **Custo baixo:** poucos recursos de infraestrutura (uma EC2 e um RDS são suficientes).
- **Depuração simples:** um único log e um único ponto de falha para investigar.
- **Consistência:** transações e integridade ficam no próprio banco, sem consistência eventual entre serviços.
- **Menor carga operacional:** não exige equipe grande nem plataforma de microsserviços.

2.3 Desvantagens

- **Escala limitada e pouco granular:** é preciso replicar tudo para escalar qualquer parte.
- **Deploys de maior risco:** uma alteração pequena reimplanta a aplicação inteira, e um bug pode derrubar todas as funções.
- **Acoplamento tecnológico:** toda a aplicação fica presa a uma linguagem, um framework e um banco.
- **Crescimento do código:** com o tempo, o `app.py` tende a ficar grande e difícil de manter, com vários times alterando os mesmos arquivos.
- **Disponibilidade:** o banco e a aplicação são pontos únicos de falha.
- **Pontos de atenção atuais:** a API não tem autenticação, as respostas de erro 500 expõem `str(e)` e não há testes automatizados.

2.4 Conclusão

Para um MVP, o monolito é uma escolha adequada: entrega valor rápido, com baixo custo e baixa complexidade. A evolução para serviços separados (por exemplo, separar autenticação, avaliação de flags e administração) só se justifica quando houver necessidade real de escala independente, de times autônomos ou de ciclos de deploy distintos. Até lá, vale organizar o código em módulos para facilitar essa separação futura.

3. Análise dos 12 Fatores (12-Factor App)

3.1 Tabela de aderência

#	Fator	Situação	Evidência
1	Codebase	Atende	Um único repositório, mesmo código em local e AWS

#	Fator	Situação	Evidência
2	Dependências	Atende	Versões fixadas no `requirements.txt` (Flask, Werkzeug, psycopg2-binary, gunicorn); o `import click` do `app.py` não está declarado e só funciona por ser dependência do Flask
3	Configuração	Atende	`DB_HOST`, `DB_NAME`, `DB_USER` e `DB_PASSWORD` vêm de variáveis de ambiente, sem credencial no código
4	Serviços de apoio	Atende	O PostgreSQL é um recurso externo, trocável por RDS apenas alterando `DB_HOST`
5	Build, release, run	Parcial	O `init-db` roda no entrypoint a cada start, misturando release e run; em dev, o volume `.:/app` sobrepõe o código construído na imagem
6	Processos	Atende	Aplicação sem estado local; todo o estado fica no banco
7	Vínculo de porta	Parcial	A aplicação expõe a porta 5000 sozinha (Gunicorn), mas ela está fixa no `CMD` e no `app.run`, sem ler `PORT`
8	Concorrência	Parcial	Escala horizontalmente por ser stateless, mas o Gunicorn usa o padrão de 1 worker e cada requisição abre uma nova conexão com o banco, sem pool
9	Descartabilidade	Parcial	Inicia rápido, mas o `init_db` engole os erros e a aplicação pode subir sem a tabela
10	Paridade dev/prod	Parcial	Em dev o código vem do volume e em prod viria da imagem; credenciais fixas (`user` / `password`); Postgres 13 e Python 3.9 já fora de suporte
11	Logs	Parcial	Saída em stdout (correto) e `PYTHONUNBUFFERED=1`, mas apenas com `print`, sem módulo `logging` nem níveis de log
12	Processos administrativos	Atende	Comando pontual `flask init-db`

Resumo: **6 fatores atendem plenamente** (1, 2, 3, 4, 6 e 12) e **6 atendem parcialmente** (5, 7, 8, 9, 10 e 11). Nenhum fator está totalmente descumprido.

3.2 Ajustes necessários para produção

Prioridade	Ajuste	Fator relacionado
Alta	Remover o volume `.:/app` ao implantar; usar o código da imagem	5, 10
Alta	Não usar credenciais fixas; passar as do RDS por variáveis de ambiente ou gerenciador de segredos, fora do repositório	3, 10
Alta	Adicionar autenticação à API	Segurança
Alta	Não devolver `str(e)` nas respostas de erro 500; registrar o detalhe apenas no log	Segurança, 11
Média	Separar a inicialização do banco (release) da subida da aplicação (run), ou garantir que o `init_db` falhe de forma explícita	5, 9
Média	Ler `DB_PORT` no `app.py` (hoje é definida no compose mas ignorada)	3
Média	Ler a porta de `PORT` e configurar `--workers` no Gunicorn	7, 8
Média	Usar pool de conexões ao banco	8
Média	Trocar `print` por `logging` com níveis	11
Baixa	Atualizar Python e PostgreSQL para versões com suporte e usar a mesma versão em dev e RDS	10
Baixa	Declarar `click` no `requirements.txt`	2
Baixa	No Dockerfile: executar como usuário não-root, mover o `apt-get install` para antes do `COPY app.py` e usar `ENV chave=valor`	Boas práticas

3.3 Considerações para a AWS (EC2 + RDS)

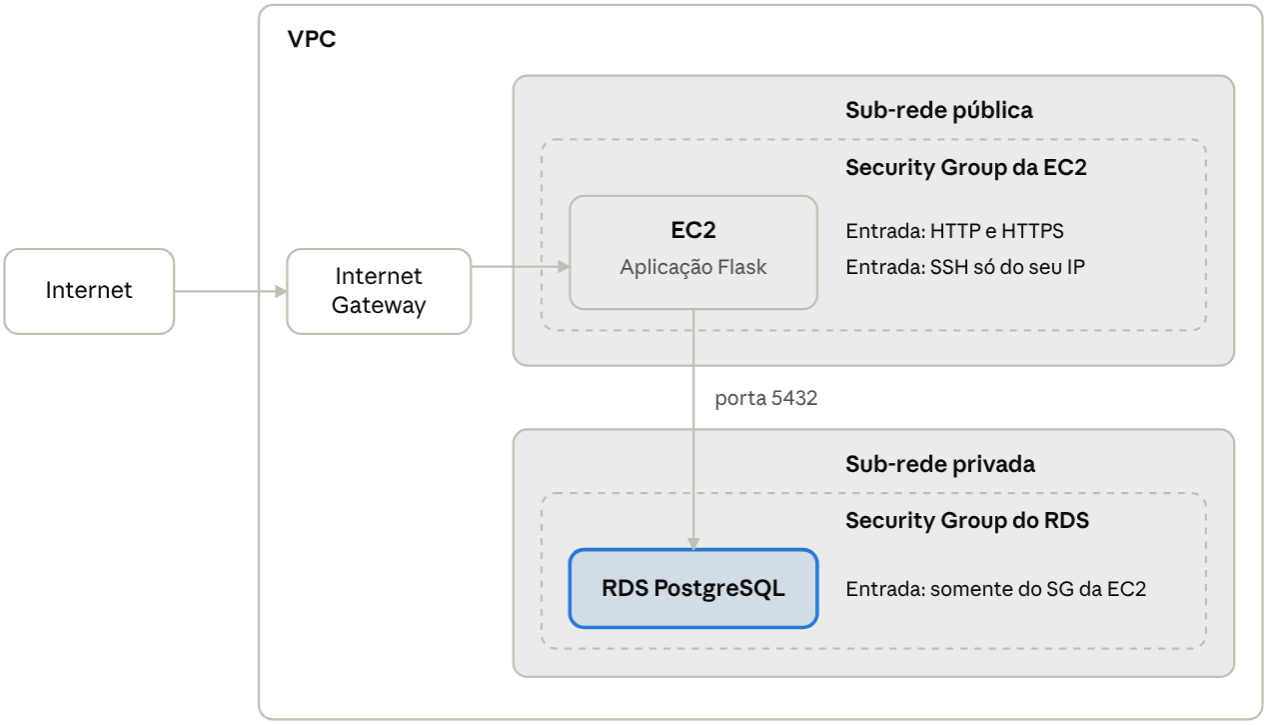
- Definir `DB_HOST`, `DB_NAME`, `DB_USER` e `DB_PASSWORD` na EC2, com `DB_HOST` apontando para o endpoint do RDS.

- No Security Group do RDS, liberar a porta 5432 apenas a partir do Security Group da EC2.
- No Security Group da EC2, liberar a porta 5000 apenas para IPs necessários, enquanto a API não tiver autenticação.
- Escolher no RDS uma versão do PostgreSQL com suporte (por exemplo, 16, confirmando no console) e alinhar a imagem do compose local.

4. Arquitetura Cloud na AWS

A aplicação roda em uma EC2 na sub-rede pública, e o banco RDS fica em uma sub-rede privada, acessível só pela EC2.

O RDS na sub-rede privada só aceita conexões da EC2



arquitetura AWS · VPC, EC2, RDS e 2 Security Groups

O tráfego entra pela internet, passa pelo Internet Gateway e chega à EC2. A EC2 acessa o RDS pelo endpoint, configurado em `DB_HOST`.

4.1 Componentes

Componente	Função
VPC	Rede isolada que contém todos os recursos
Sub-rede pública	Abriga a EC2; tem rota para o Internet Gateway

Componente	Função
Internet Gateway	Conecta a VPC à internet
EC2	Executa a aplicação Flask
Sub-rede privada	Abriga o RDS; sem rota direta para a internet
RDS PostgreSQL	Banco de dados da aplicação, sem acesso público

4.2 Security Groups

Security Group	Direção	Protocolo e porta	Origem
EC2	Entrada	HTTP, 80	Qualquer origem (0.0.0.0/0)
EC2	Entrada	HTTPS, 443	Qualquer origem (0.0.0.0/0)
EC2	Entrada	SSH, 22	Somente o seu IP (/32)
RDS	Entrada	PostgreSQL, 5432	Security Group da EC2

4.3 Observações

- A aplicação escuta na porta 5000. Para atender em HTTP (80), mapeie a porta (por exemplo, `80:5000` no Docker) ou use um proxy reverso; o HTTPS (443) exige certificado e proxy. Em testes, é possível liberar a 5000 só para o seu IP.
- O RDS exige um DB subnet group com sub-redes em pelo menos 2 zonas de disponibilidade. Crie 2 sub-redes privadas, mesmo usando uma única instância.
- No RDS, deixe *Public access* desativado; a EC2 conecta pelo endpoint do RDS.
- A origem da regra do RDS é o Security Group da EC2, e não um IP, o que mantém a regra válida se a EC2 for recriada.

5. AWS prático: provisionamento e execução

Esta seção documenta, na ordem do enunciado, o que foi feito no console da AWS. Toda a infraestrutura foi criada manualmente na região **N. Virginia (us-east-1)**, seguindo o diagrama da seção 4.

5.1 Provisionamento da infraestrutura

VPC. Criada pelo assistente *VPC and more*: `togglemaster-vpc` (`10.0.0.0/16`), 2 zonas de disponibilidade (`us-east-1a` e `us-east-1b`), 2 sub-redes públicas e 2 sub-redes privadas, sem NAT Gateway (não é necessário aqui e tem custo por hora). O assistente cria também o Internet Gateway e as tabelas de rota.

Security Groups, criados na `togglemaster-vpc`:

Security Group	Entrada	Origem
`SG-ec2`	HTTP (80)	Qualquer origem (0.0.0.0/0)
`SG-ec2`	HTTPS (443)	Qualquer origem (0.0.0.0/0)
`SG-ec2`	SSH (22)	Somente o IP do usuário (/32)
`SG-ec2`	TCP 5000 (porta da aplicação)	Somente o IP do usuário (/32)
`SG-rds`	PostgreSQL (5432)	Security Group `SG-ec2`

A AWS não aceita nomes de Security Group que começam com `sg-` (prefixo reservado aos IDs), por isso os nomes usam `SG-` em maiúsculas.

RDS PostgreSQL, na sub-rede privada:

Configuração	Valor
DB subnet group	`togglemaster-subnet-group`, com as 2 sub-redes privadas (`us-east-1a` e `us-east-1b`)
Engine	PostgreSQL
Identificador	`togglemaster-db`
Classe	`db.t3.micro`
Banco inicial	`togglemaster`
Usuário master	`postgres`
Senha	Definida manualmente (*Self managed*); fornecida à aplicação por variável de ambiente
Acesso público	Não (*Publicly accessible: No*)
Security Group	`SG-rds`
Porta	5432

EC2, na sub-rede pública:

Configuração	Valor
Nome	`togglemaster-app`
AMI	Amazon Linux 2023
Tipo	`t3.micro`
Sub-rede	`togglemaster-subnet-public1-us-east-1a`
IP público	Habilitado (`3.208.87.115`)
Security Group	`SG-ec2`
Key pair	`togglemaster-key` (RSA, formato `.pem`)

5.2 Conexão à EC2 via SSH

No Git Bash, na pasta onde está o arquivo `.pem` :

```
chmod 400 togglemaster-key.pem
ssh -i togglemaster-key.pem ec2-user@3.208.87.115
```

O prompt mudou para `[ec2-user@ip-10-0-0-71 ~]$` , confirmando o acesso à instância.

5.3 Instalação das dependências e do código-fonte

Na EC2, foram instalados o Git, o pip e o cliente `psql` (para testar a conexão com o RDS). O código veio do fork no GitHub, e as dependências ficaram em um ambiente virtual:

```
sudo dnf update -y
sudo dnf install -y git python3-pip postgresql15
git clone https://github.com/MarleyRacolto/toggle-master-monolith.git
cd toggle-master-monolith
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

5.4 Configuração da conexão com o RDS

A aplicação lê as credenciais de variáveis de ambiente (fator 3 do 12-Factor):

Variável	Valor
`DB_HOST`	Endpoint do RDS: `togglemaster-db.c2t0k6wg2d3k.us-east-1.rds.amazonaws.com`
`DB_NAME`	`togglemaster`
`DB_USER`	`postgres`
`DB_PASSWORD`	Digitada no terminal, sem ficar no histórico nem no repositório

```
export DB_HOST="togglemaster-db.c2t0k6wg2d3k.us-east-1.rds.amazonaws.com"
export DB_NAME="togglemaster"
export DB_USER="postgres"
read -s -p "Senha do RDS: " DB_PASSWORD; export DB_PASSWORD
```

Teste de conectividade EC2 → RDS antes de subir a aplicação:

```
PGPASSWORD="$DB_PASSWORD" psql -h "$DB_HOST" -U "$DB_USER" -d "$DB_NAME" -c
"select 1;"
```

O resultado foi `(1 row)`, comprovando que a EC2 alcança o RDS na sub-rede privada.

5.5 Execução da aplicação

```
export FLASK_APP=app.py
flask init-db
nohup gunicorn --bind 0.0.0.0:5000 app:app > app.log 2>&1 &
```

O `flask init-db` respondeu `Tabela 'flags' inicializada com sucesso.`, e o Gunicorn passou a rodar em segundo plano na porta 5000.

5.6 Validação do acesso público

Do computador local (fora da EC2), usando o IP público da instância:

```
curl http://3.208.87.115:5000/health
curl -X POST http://3.208.87.115:5000/flags -H "Content-Type:
application/json" -d '{"name":"nova-tela","is_enabled":true}'
curl http://3.208.87.115:5000/flags
```

Requisição	Resposta
`GET /health`	`{"status":"ok"}`
`POST /flags`	`{"message":"Flag 'nova-tela' criada com sucesso"}`
`GET /flags`	`[{"is_enabled":true,"name":"nova-tela"}]`

O resultado comprova o caminho completo: internet, Internet Gateway, EC2 na sub-rede pública e RDS na sub-rede privada, com gravação e leitura de dados.

5.7 Problemas encontrados e soluções

Problema	Causa	Solução
O assistente de criação do banco vinha com *Aurora (PostgreSQL Compatible)* e *Express configuration*	Esse modo cria o banco sem VPC, com acesso pela internet, o contrário do diagrama	Selecionar **PostgreSQL** e **Full configuration**, escolhendo a VPC, o subnet group e o `SG-rds`
Nome `sg-rds` recusado	Nomes de Security Group não podem começar com `sg-`	Usar `SG-rds` e `SG-ec2`
A EC2 foi criada com o Security Group `default`	A opção *Create security group* estava ativa no assistente	**Actions > Security > Change security groups**: remover o `default` e associar o `SG-ec2`
`curl` ao IP público, executado dentro da EC2, ficava sem resposta	A porta 5000 só aceita o IP do usuário, e não a própria instância	Executar o teste do computador local; dentro da EC2, usar `localhost`

5.8 Custos e encerramento

- EC2 e RDS cobram por hora enquanto existirem. Após a entrega, devem ser parados ou apagados: EC2 em **Instance state > Stop instance** (ou *Terminate*), RDS em **Actions > Stop temporarily** (ou *Delete*).
- O RDS parado é religado automaticamente pela AWS após 7 dias, e o armazenamento continua sendo cobrado enquanto ele existir.
- Ao religar a EC2, o IP público muda e a aplicação precisa ser iniciada de novo (Gunicorn e variáveis de ambiente não persistem).

- VPC, sub-redes e Security Groups não têm custo.
- A porta 5000 está aberta só para o IP do usuário, porque a API não tem autenticação.

5.9 Checklist do exercício

Item do enunciado	Situação
Provisionar manualmente a infraestrutura desenhada	Feito: VPC, sub-redes públicas e privadas, EC2, RDS e 2 Security Groups
Conectar-se à EC2 via SSH	Feito
Instalar as dependências e o código-fonte	Feito: Git, pip, ambiente virtual e `requirements.txt`
Configurar a conexão com o RDS por variáveis de ambiente	Feito: `DB_HOST`, `DB_NAME`, `DB_USER` e `DB_PASSWORD`
Executar a aplicação e validar o acesso público pelo IP	Feito: `/health`, criação e listagem de flag pelo IP público

6. Conclusão geral

A aplicação roda localmente com um único comando e cumpre bem o papel de MVP. Atende ou atende parcialmente a todos os 12 fatores, e os principais ajustes para um ambiente de produção estão em configuração e segredos, separação entre release e run, observabilidade (logs) e segurança da API. A infraestrutura desenhada foi provisionada manualmente na AWS (VPC, EC2 e RDS PostgreSQL em sub-rede privada), e a aplicação foi validada pelo IP público da EC2, gravando e lendo dados no RDS.